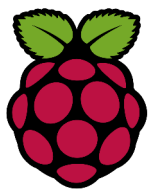


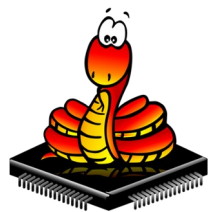


Atelier Raspi

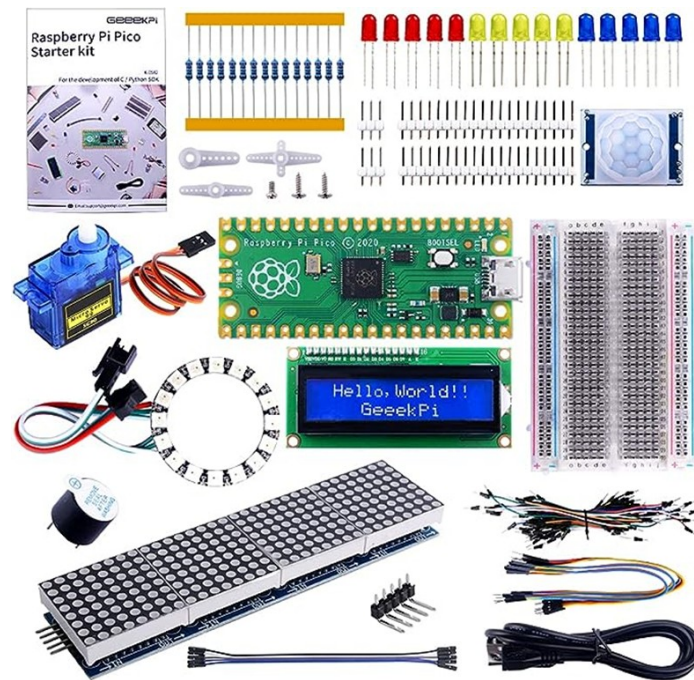
Atelier N°20 La Base de Jeux



Logo du Raspberry Pico



Logo du MicroPython



L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

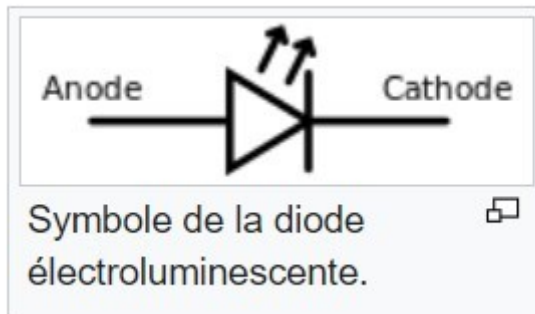
La LED



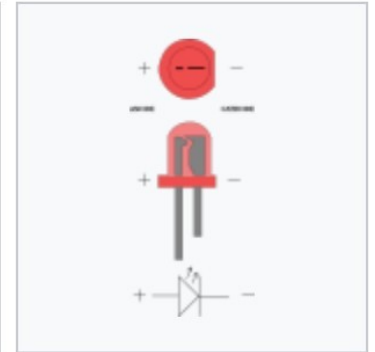
Une LED (Light Emitting Diode) ou DEL (Diode ELectroluminescente) est un composant électronique qui produit de la lumière lorsqu'un courant électrique le traverse. Elle ne laisse passer le courant électrique que dans un seul sens, de l'anode (A) vers la cathode (K). Il faut donc faire attention à son sens de branchement.

Pour protéger la LED, il faut toujours lui associer, en série, une résistance qui permet de limiter le courant qui la traverse. En effet, une trop forte intensité (supérieure à 50 mA pour des LEDs standard de diamètre 5mm) provoque sa destruction.

La valeur ohmique de cette résistance doit être calculée afin que l'intensité qu'elle laisse passer dans la LED soit suffisante pour assurer un éclairage optimal de celle-ci : 20 mA pour des LEDs standard de diamètre 5mm.

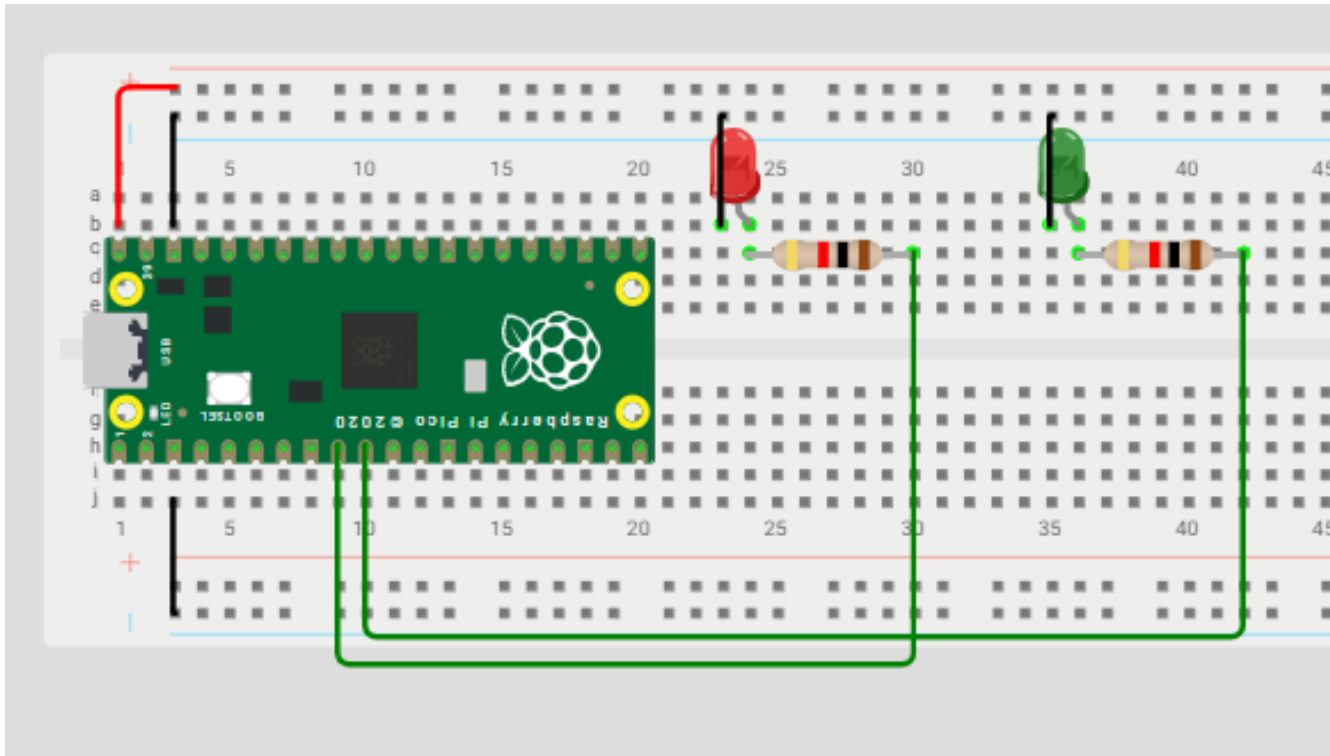


Gros-plan d'une diode électroluminescente.



L'anode et la cathode d'une LED. Les signes indiquent la polarisation (courant conventionnel) lorsque la diode est utilisée en sens direct.

Câblage des 2 LEDs



Composant	GPIO
Led 1 (rouge)	6
Led 2 (verte)	7

Code couleur des câbles :

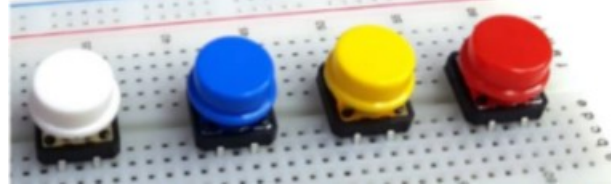
- Vcc : Rouge / Orange
- Gnd : Noir / Marron
- Données : Vert / Bleu / Jaune / Violet

Test des LEDs

```
1 # on a besoin des fonctions "time" et "Pin" (qui est dans "machine") donc on les importe
2 import time
3 from machine import Pin
4
5 #On choisit le Pin sur lequel on va monter la LED et on lui dit que c'est une sortie
6 #(pas une entree de signal) c'est a dire que la sortie PIN va fournir l'energie pour commander la LED
7 LED = Pin(6, Pin.OUT)
8 #On donne la valeur 1 à la sortie pour allumer la LED
9 LED.value(1)
10 #On lui dit de rester allumée pendant 3 secondes
11 time.sleep(3)
12 #On lui dit de s'eteindre
13 LED.value(0)
14 #On lui dit de rester éteinte pendant 2 secondes
15 time.sleep(2)
16
```



Le bouton poussoir



Un **bouton poussoir** est un interrupteur (ou contacteur) **monostable** : il retourne seul dans la position repos (« relâché »).

Il peut être :

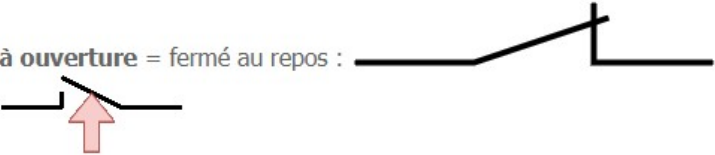
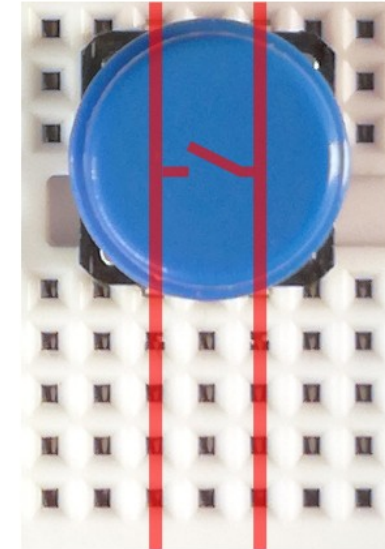
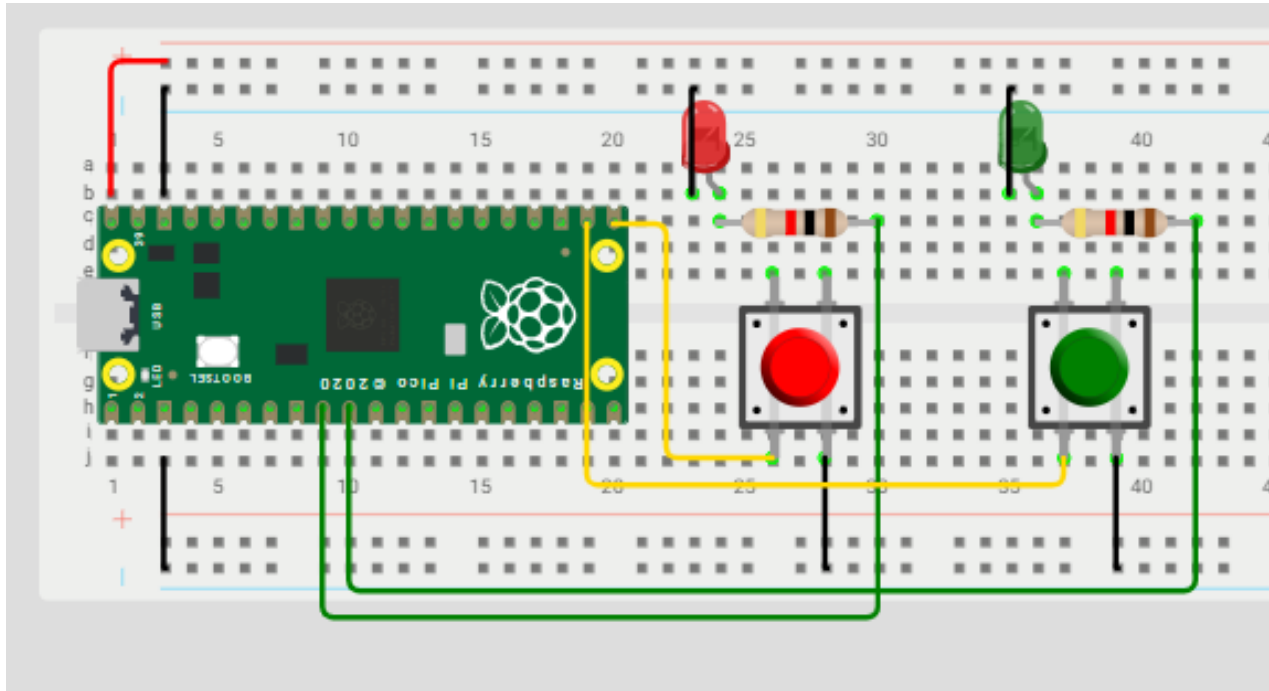
- à **fermeture** = ouvert au repos : 
- à **ouverture** = fermé au repos : 



Schéma électrique interne :



Câblage des 2 BPs

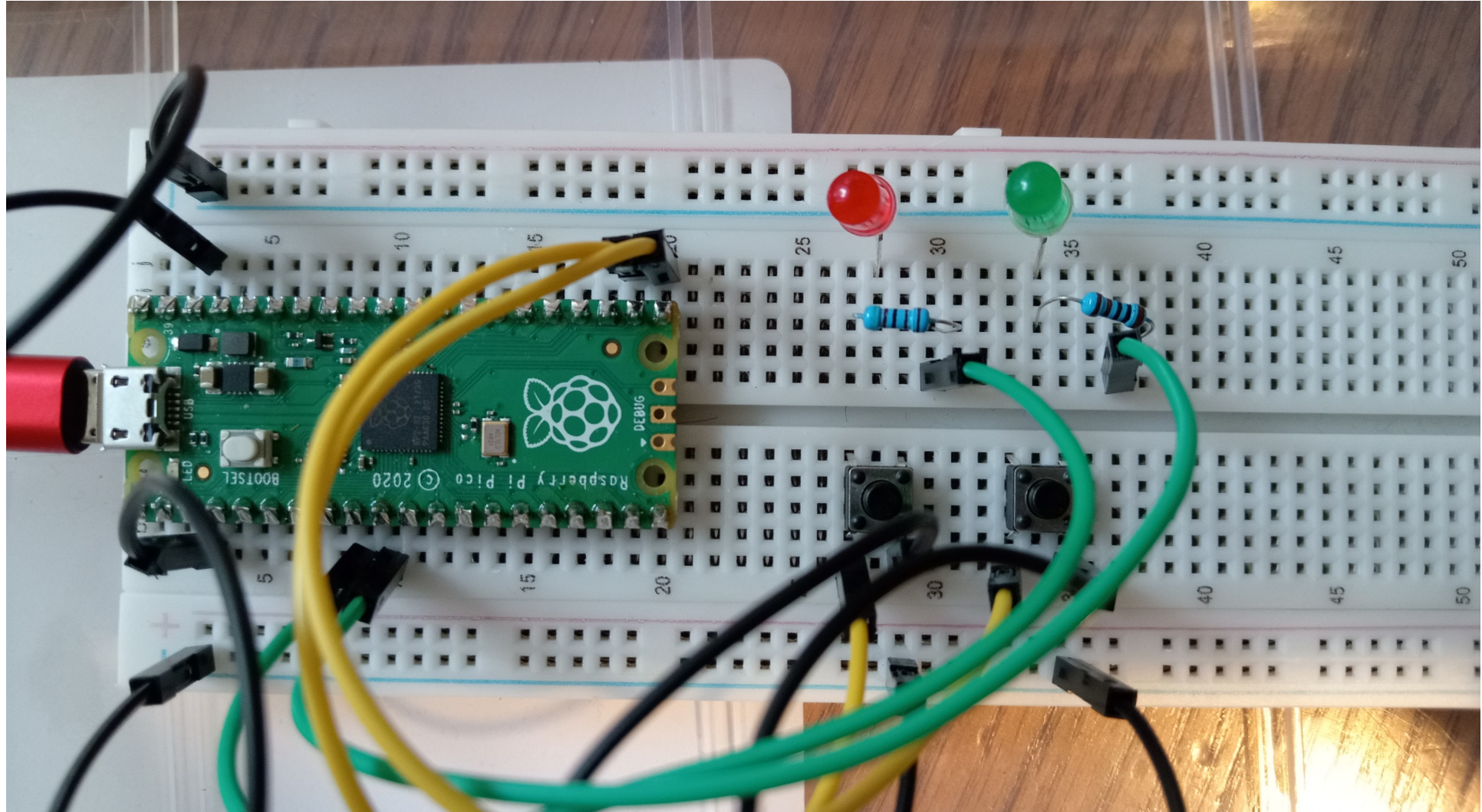


Composant	GPIO
BP 1 (rouge)	16
BP 2 (vert)	17

Test des BPs

```
1 from machine import Pin
2 import time
3
4 bouton = Pin(16, machine.Pin.IN, Pin.PULL_UP)
5
6 while True:
7     if bouton.value() == 0:
8         print("Bouton Appuyé")
9     else:
10        print("Bouton Relaché")
11        time.sleep_ms(50)
12 |
```

Montage final



Le ruban Neopixel

Le ruban NeoPixel

Une LED... mais beaucoup plus évoluée !
Un NeoPixel est une LED RGB programmable.

Contrairement à une LED classique, on peut choisir :

- sa couleur ;
- sa luminosité ;
- quelle LED du ruban doit s'allumer.

Notre ruban contient 2 NeoPixels.

Ruban NeoPixel

LED 0 → 

LED 1 → 

Chaque LED possède un numéro

Le programme peut commander chaque LED individuellement.

Pour notre ruban :



Par exemple :

 Python

```
leds.set_pixel(0, couleur_rouge)
```

→ La LED 0 devient rouge.

 Python

```
leds.set_pixel(1, couleur_bleu)
```

→ La LED 1 devient bleue.

RGB (255, 0, 0)
RGB (255, 127, 0)
RGB (255, 255, 0)
RGB (0, 255, 0)
RGB (0, 0, 255)
RGB (75, 0, 130)

Une NeoPixel peut afficher toutes sortes de couleurs

Chaque couleur est composée de trois valeurs :

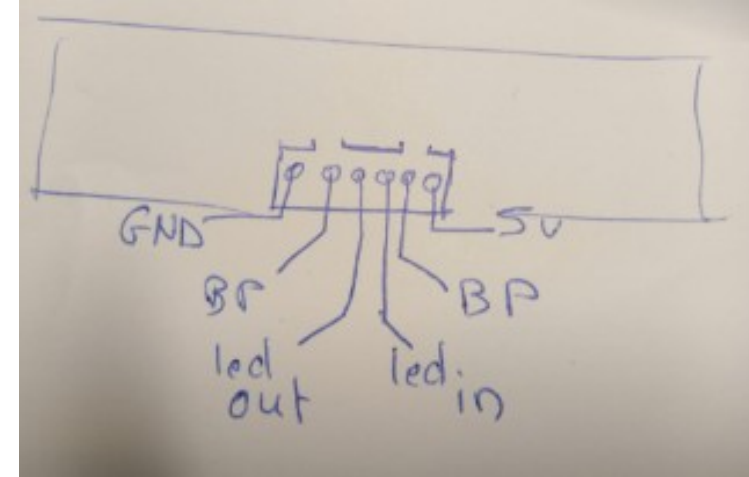
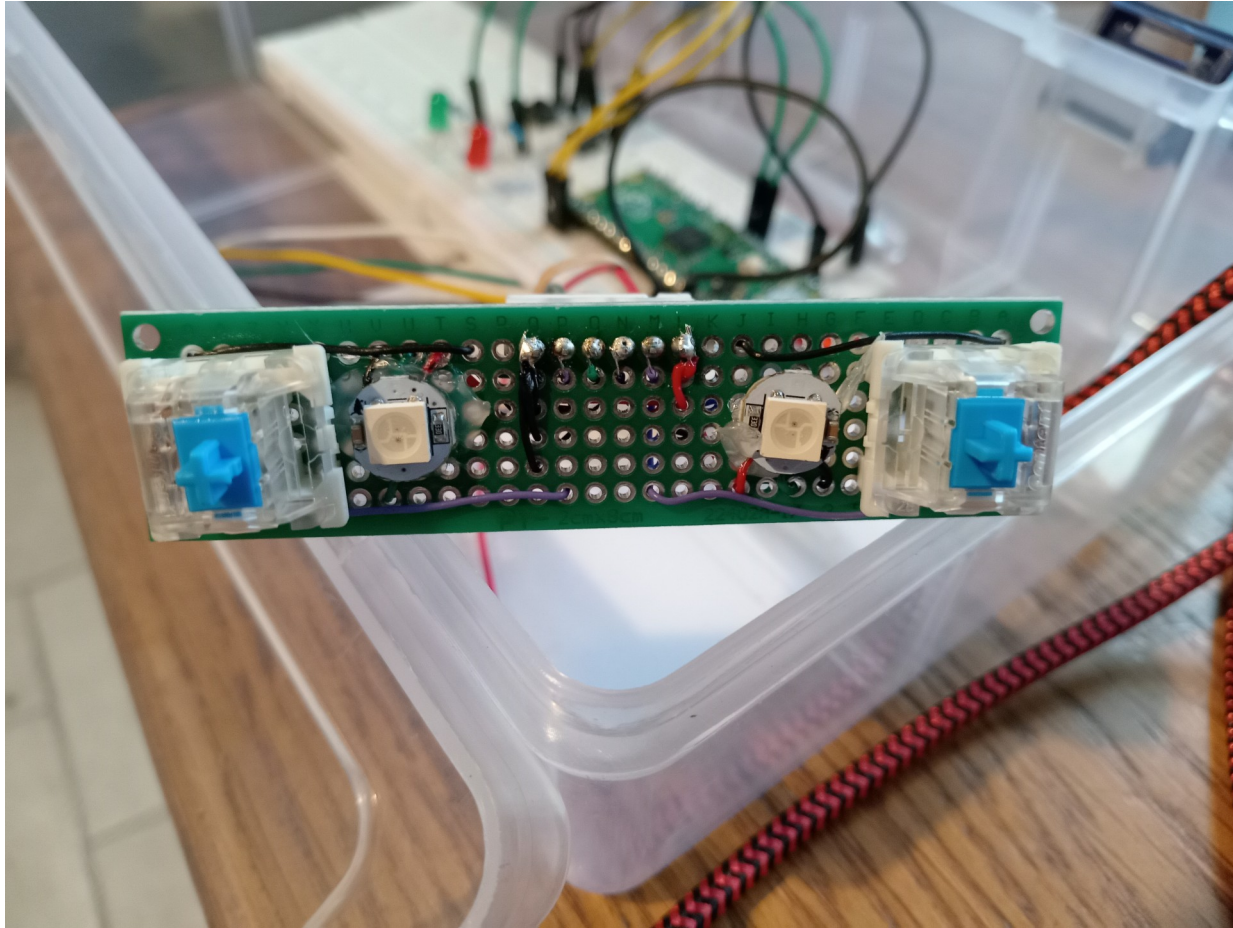
Rouge – Vert – Bleu

On appelle cela le système RGB.

(255, 0, 0)	→  Rouge
(0, 255, 0)	→  Vert
(0, 0, 255)	→  Bleu
(255, 255, 0)	→  Jaune
(255, 0, 255)	→  Violet
(0, 0, 0)	→  Éteint

Chaque valeur peut aller de 0 à 255.

Câblage du module Bouton + Neopixel



Composant	GPIO
BP 3 (fil noir)	18
BP 4 (fil vert)	19
Neopixel (fil jaune)	13

Test du « ruban » Neopixel

```
1 from neopixel import Neopixel
2 import time
3
4 # declaration du ruban de leds
5 NUM_LED = 2
6 leds = Neopixel(NUM_LED, 0, 13, "GRB")
7
8 # on definit la luminosite des leds / 255
9 leds.brightness(15)
10 # on eteint toute les leds
11 leds.clear()
12 # on affiche
13 leds.show()
14
15
16 while True:
17     # on allume toutes les leds en rouge pendant 1 sec
18     leds.fill((255,0,0))
19     leds.show()
20     time.sleep(1)
21     # on allume toutes les leds en vert pendant 1 sec
22     leds.fill((0,255,0))
23     leds.show()
24     time.sleep(1)
25     # on allume toutes les leds en bleu pendant 1 sec
26     leds.fill((0,0,255))
27     leds.show()
28     time.sleep(1)
29
```

```
1 from neopixel import Neopixel
2 import time
3
4 # declaration du ruban de leds
5 NUM_LED = 2
6 leds = Neopixel(NUM_LED, 0, 13, "GRB")
7
8 # declaration de quelques couleurs
9 couleur_rouge = (255,0,0)
10 couleur_vert = (0,255,0)
11 couleur_bleu = (0,0,255)
12 couleur_noir = (0,0,0)
13
14 # on definit la luminosite des leds
15 leds.brightness(15)
16 # on eteint toute les leds
17 leds.clear()
18 # on affiche
19 leds.show()
20
21 while True:
22     # on allume tour a tour les leds en bleu
23     for i in range(0, NUM_LED):
24         leds.set_pixel(i, couleur_bleu)
25         leds.show()
26         time.sleep(1)
27     time.sleep(1)
28     # on allume tour a tour les leds en vert
29     for i in range(0, NUM_LED):
30         leds.set_pixel(i, couleur_vert)
31         leds.show()
32         time.sleep(1)
33     time.sleep(1)
34     #leds.clear()
35
```

Le Jeu de Simon simplifié – Version LEDs

🎮 Jeu 1 — Le Simon à 2 couleurs

Le principe

Le jeu mémorise une séquence de couleurs et l'allonge à chaque niveau.

1. 💡 Une LED s'allume : rouge ou verte
2. 🧠 Le joueur doit mémoriser la séquence.
3. 🔄 Il reproduit la séquence avec les deux boutons.
4. ✅ S'il réussit, une nouvelle couleur est ajoutée.
5. ❌ S'il se trompe, la partie s'arrête et le score est affiché.
6. 🔄 Une nouvelle partie recommence avec une séquence vide.

💻 Ce que l'on va apprendre

Avec ce jeu, on découvre notamment :

- les listes pour mémoriser la séquence ;
- les boucles `for` et `while` ;
- les conditions `if / else` ;
- les fonctions pour organiser le programme ;
- le module `random` pour choisir une couleur au hasard ;
- la lecture de l'état d'un bouton ;
- le pilotage des LEDs ;
- la gestion d'un niveau et d'un score.

🌟 Une idée importante

Le programme est découpé en petites fonctions :

```
allumer_led()
↓
attendre_bouton()
↓
afficher_sequence()
↓
JEU SIMON
```



Pourquoi faire des fonctions ?

Pour éviter de répéter le même code et rendre le programme **plus facile à comprendre et à modifier**.

🚀 Pour aller plus loin

Certaines fonctions du jeu sont volontairement laissées vides :

```
</> Python

def erreur():
    pass

def victoire():
    pass
```

À vous de les programmer !

Par exemple, que pourrait faire le jeu lorsqu'un joueur réussit un niveau ?
Et lorsqu'il se trompe ?

Le Jeu de Simon simplifié – Version Neopixel

Évolution du Simon : les LEDs deviennent des NeoPixels

Même jeu, nouvelles possibilités !

Dans la première version, le jeu utilise :

- 2 LEDs classiques
- 2 boutons
- 1 couleur par LED

Dans la nouvelle version, les deux LEDs sont remplacées par 2 LEDs RGB NeoPixel.

Chaque LED du ruban peut être commandée **indépendamment** et peut afficher différentes couleurs.

Ce qui change

Version 1

GPIO 6 →  LED rouge
GPIO 7 →  LED verte

Version 2

GPIO 13 →  NeoPixel 1
 NeoPixel 2

Un seul GPIO permet de commander les deux NeoPixels.

Le jeu reste identique

Le joueur doit toujours :

1. regarder la séquence lumineuse ;
2. mémoriser les couleurs ;
3. reproduire la séquence avec les boutons ;
4. progresser niveau après niveau ;
5. recommencer s'il se trompe.

La logique du jeu ne change donc presque pas.

Ce sont principalement les fonctions qui contrôlent les LEDs qu'il faut modifier.

Le jeu de reflexe – Version LEDs

Duel de réflexes

Le principe

Une LED s'allume au hasard :

● LED rouge → appuyer sur le bouton rouge

● LED verte → appuyer sur le bouton vert

Le joueur doit être le plus rapide possible.

Le programme mesure son temps de réaction.



Objectif : obtenir le temps de réaction le plus faible.

Ce que l'on va apprendre

- les listes pour regrouper les LEDs et les boutons ;
- les boucles for et while ;
- les conditions if / else ;
- les fonctions pour organiser le programme ;
- le module random pour choisir au hasard ;
- la lecture de l'état d'un bouton ;
- la mesure du temps avec ticks_ms() et ticks_diff() ;
- le pilotage des LEDs.

Une idée importante

Le programme utilise plusieurs fonctions :

```
eteindre_leds()
attendre_bouton()
random.randint(0, 1)
time.time()
time.time_diff(...)
```

Pourquoi faire des fonctions ?

Pour éviter de répéter le même code et rendre le programme plus facile à comprendre et à modifier.

Pour aller plus loin

Le jeu peut être amélioré.

Quelques idées :

- mesurer le meilleur temps ;
- compter le nombre de bonnes réponses ;
- ajouter un nombre maximum de manches ;
- afficher un message selon le résultat ;
- détecter un appui trop tôt, avant l'allumage.

À vous de programmer une amélioration !

Par exemple :
Que pourrait faire le programme si le joueur appuie sur le mauvais bouton ?